```
/**
 * Mergesort algorithm (driver).
 */
template <typename Comparable>
void mergeSort( vector<Comparable> & a )
{
    vector<Comparable> tmpArray( a.size( ) );

    mergeSort( a, tmpArray, 0, a.size( ) - 1 );
}

/**
 * Internal method that makes recursive calls.
 * a is an array of Comparable items.
 * tmpArray is an array to place the merged result.
 * left is the left-most index of the subarray.
 * right is the right-most index of the subarray.
 */
template <typename Comparable>
void mergeSort( vector<Comparable> & a,
                vector<Comparable> & tmpArray, int left, int right )
{
    if( left < right )
    {
        int center = ( left + right ) / 2;
        mergeSort( a, tmpArray, left, center );
        mergeSort( a, tmpArray, center + 1, right );
        merge( a, tmpArray, left, center + 1, right );
    }
}
```

Figure 7.11 Mergesort routines

temporary array active at any point, and that the temporary array can be created in the public `mergeSort` driver. Further, we can use any part of the temporary array; we will use the same portion as the input array `a`. This allows the improvement described at the end of this section. Figure 7.12 implements the `merge` routine.

7.6.1 Analysis of Mergesort

Mergesort is a classic example of the techniques used to analyze recursive routines: We have to write a recurrence relation for the running time. We will assume that N is a power of 2 so that we always split into even halves. For $N = 1$, the time to mergesort is constant, which we will denote by 1. Otherwise, the time to mergesort N numbers is equal to the

```
/**
 * Internal method that merges two sorted halves of a subarray.
 * a is an array of Comparable items.
 * tmpArray is an array to place the merged result.
 * leftPos is the left-most index of the subarray.
 * rightPos is the index of the start of the second half.
 * rightEnd is the right-most index of the subarray.
 */
template <typename Comparable>
void merge( vector<Comparable> & a, vector<Comparable> & tmpArray,
            int leftPos, int rightPos, int rightEnd )
{
    int leftEnd = rightPos - 1;
    int tmpPos = leftPos;
    int numElements = rightEnd - leftPos + 1;

    // Main loop
    while( leftPos <= leftEnd && rightPos <= rightEnd )
        if( a[ leftPos ] <= a[ rightPos ] )
            tmpArray[ tmpPos++ ] = std::move( a[ leftPos++ ] );
        else
            tmpArray[ tmpPos++ ] = std::move( a[ rightPos++ ] );

    while( leftPos <= leftEnd )    // Copy rest of first half
        tmpArray[ tmpPos++ ] = std::move( a[ leftPos++ ] );

    while( rightPos <= rightEnd )  // Copy rest of right half
        tmpArray[ tmpPos++ ] = std::move( a[ rightPos++ ] );

    // Copy tmpArray back
    for( int i = 0; i < numElements; ++i, --rightEnd )
        a[ rightEnd ] = std::move( tmpArray[ rightEnd ] );
}
```

Figure 7.12 `merge` routine

time to do two recursive mergesorts of size $N/2$, plus the time to merge, which is linear. The following equations say this exactly:

$$T(1) = 1$$
$$T(N) = 2T(N/2) + N$$

This is a standard recurrence relation, which can be solved several ways. We will show two methods. The first idea is to divide the recurrence relation through by N. The reason for doing this will become apparent soon. This yields

$$\frac{T(N)}{N} = \frac{T(N/2)}{N/2} + 1$$

This equation is valid for any N that is a power of 2, so we may also write

$$\frac{T(N/2)}{N/2} = \frac{T(N/4)}{N/4} + 1$$

and

$$\frac{T(N/4)}{N/4} = \frac{T(N/8)}{N/8} + 1$$
$$\vdots$$
$$\frac{T(2)}{2} = \frac{T(1)}{1} + 1$$

Now add up all the equations. This means that we add all of the terms on the left-hand side and set the result equal to the sum of all of the terms on the right-hand side. Observe that the term $T(N/2)/(N/2)$ appears on both sides and thus cancels. In fact, virtually all the terms appear on both sides and cancel. This is called **telescoping** a sum. After everything is added, the final result is

$$\frac{T(N)}{N} = \frac{T(1)}{1} + \log N$$

because all of the other terms cancel and there are $\log N$ equations, and so all the 1s at the end of these equations add up to $\log N$. Multiplying through by N gives the final answer.

$$T(N) = N \log N + N = O(N \log N)$$

Notice that if we did not divide through by N at the start of the solutions, the sum would not telescope. This is why it was necessary to divide through by N.

An alternative method is to substitute the recurrence relation continually on the right-hand side. We have

$$T(N) = 2T(N/2) + N$$

Since we can substitute $N/2$ into the main equation,

$$2T(N/2) = 2(2(T(N/4)) + N/2) = 4T(N/4) + N$$

we have

$$T(N) = 4T(N/4) + 2N$$

Again, by substituting $N/4$ into the main equation, we see that

$$4T(N/4) = 4(2T(N/8) + N/4) = 8T(N/8) + N$$

So we have

$$T(N) = 8T(N/8) + 3N$$

Continuing in this manner, we obtain

$$T(N) = 2^k T(N/2^k) + k \cdot N$$

Using $k = \log N$, we obtain

$$T(N) = NT(1) + N \log N = N \log N + N$$

The choice of which method to use is a matter of taste. The first method tends to produce scrap work that fits better on a standard $8^1/_2 \times 11$ sheet of paper leading to fewer mathematical errors, but it requires a certain amount of experience to apply. The second method is more of a brute-force approach.

Recall that we have assumed $N = 2^k$. The analysis can be refined to handle cases when N is not a power of 2. The answer turns out to be almost identical (this is usually the case).

Although mergesort's running time is $O(N \log N)$, it has the significant problem that merging two sorted lists uses linear extra memory. The additional work involved in copying to the temporary array and back, throughout the algorithm, slows the sort considerably. This copying can be avoided by judiciously switching the roles of `a` and `tmpArray` at alternate levels of the recursion. A variant of mergesort can also be implemented nonrecursively (Exercise 7.16).

The running time of mergesort, when compared with other $O(N \log N)$ alternatives, depends heavily on the relative costs of comparing elements and moving elements in the array (and the temporary array). These costs are language dependent.

For instance, in Java, when performing a generic sort (using a `Comparator`), an element comparison can be expensive (because comparisons might not be easily inlined, and thus the overhead of dynamic dispatch could slow things down), but moving elements is cheap (because they are reference assignments, rather than copies of large objects). Mergesort uses the lowest number of comparisons of all the popular sorting algorithms, and thus is a good candidate for general-purpose sorting in Java. In fact, it is the algorithm used in the standard Java library for generic sorting.

On the other hand, in classic C++, in a generic sort, copying objects can be expensive if the objects are large, while comparing objects often is relatively cheap because of the ability of the compiler to aggressively perform inline optimization. In this scenario, it might be reasonable to have an algorithm use a few more comparisons, if we can also use significantly fewer data movements. *Quicksort,* which we discuss in the next section, achieves this tradeoff and is the sorting routine that has been commonly used in C++ libraries. New C++11 move semantics possibly change this dynamic, and so it remains to be seen whether quicksort will continue to be the sorting algorithm used in C++ libraries.

7.7 Quicksort

As its name implies for C++, **quicksort** has historically been the fastest known generic sorting algorithm in practice. Its average running time is $O(N \log N)$. It is very fast, mainly due to a very tight and highly optimized inner loop. It has $O(N^2)$ worst-case performance, but this can be made exponentially unlikely with a little effort. By combining quicksort

with heapsort, we can achieve quicksort's fast running time on almost all inputs, with heapsort's $O(N \log N)$ worst-case running time. Exercise 7.27 describes this approach.

The quicksort algorithm is simple to understand and prove correct, although for many years it had the reputation of being an algorithm that could in theory be highly optimized but in practice was impossible to code correctly. Like mergesort, quicksort is a divide-and-conquer recursive algorithm.

Let us begin with the following simple sorting algorithm to sort a list. Arbitrarily choose any item, and then form three groups: those smaller than the chosen item, those equal to the chosen item, and those larger than the chosen item. Recursively sort the first and third groups, and then concatenate the three groups. The result is guaranteed by the basic principles of recursion to be a sorted arrangement of the original list. A direct implementation of this algorithm is shown in Figure 7.13, and its performance is, generally speaking, quite

```
template <typename Comparable>
void SORT( vector<Comparable> & items )
{
    if( items.size( ) > 1 )
    {
        vector<Comparable> smaller;
        vector<Comparable> same;
        vector<Comparable> larger;

        auto chosenItem = items[ items.size( ) / 2 ];

        for( auto & i : items )
        {
            if( i < chosenItem )
                smaller.push_back( std::move( i ) );
            else if( chosenItem < i )
                larger.push_back( std::move( i ) );
            else
                same.push_back( std::move( i ) );
        }

        SORT( smaller );     // Recursive call!
        SORT( larger );      // Recursive call!

        std::move( begin( smaller ), end( smaller ), begin( items ) );
        std::move( begin( same ), end( same ), begin( items ) + smaller.size( ) );
        std::move( begin( larger ), end( larger ), end( items ) - larger.size( ) );
    }
}
```

Figure 7.13 Simple recursive sorting algorithm

respectable on most inputs. In fact, if the list contains large numbers of duplicates with relatively few distinct items, as is sometimes the case, then the performance is extremely good.

The algorithm we have described forms the basis of the quicksort. However, by making the extra lists, and doing so recursively, it is hard to see how we have improved upon mergesort. In fact, so far, we really haven't. In order to do better, we must avoid using significant extra memory and have inner loops that are clean. Thus quicksort is commonly written in a manner that avoids creating the second group (the equal items), and the algorithm has numerous subtle details that affect the performance; therein lies the complications.

We now describe the most common implementation of quicksort—"classic quicksort," in which the input is an array, and in which no extra arrays are created by the algorithm.

The classic quicksort algorithm to sort an array S consists of the following four easy steps:

1. If the number of elements in S is 0 or 1, then return.
2. Pick any element v in S. This is called the **pivot.**
3. **Partition** $S - \{v\}$ (the remaining elements in S) into two disjoint groups: $S_1 = \{x \in S - \{v\} | x \leq v\}$, and $S_2 = \{x \in S - \{v\} | x \geq v\}$.
4. Return {quicksort(S_1) followed by v followed by quicksort(S_2)}.

Since the partition step ambiguously describes what to do with elements equal to the pivot, this becomes a design decision. Part of a good implementation is handling this case as efficiently as possible. Intuitively, we would hope that about half the elements that are equal to the pivot go into S_1 and the other half into S_2, much as we like binary search trees to be balanced.

Figure 7.14 shows the action of quicksort on a set of numbers. The pivot is chosen (by chance) to be 65. The remaining elements in the set are partitioned into two smaller sets. Recursively sorting the set of smaller numbers yields $0, 13, 26, 31, 43, 57$ (by rule 3 of recursion). The set of large numbers is similarly sorted. The sorted arrangement of the entire set is then trivially obtained.

It should be clear that this algorithm works, but it is not clear why it is any faster than mergesort. Like mergesort, it recursively solves two subproblems and requires linear additional work (step 3), but, unlike mergesort, the subproblems are not guaranteed to be of equal size, which is potentially bad. The reason that quicksort is faster is that the partitioning step can actually be performed in place and very efficiently. This efficiency more than makes up for the lack of equal-sized recursive calls.

The algorithm as described so far lacks quite a few details, which we now fill in. There are many ways to implement steps 2 and 3; the method presented here is the result of extensive analysis and empirical study and represents a very efficient way to implement quicksort. Even the slightest deviations from this method can cause surprisingly bad results.

7.7.1 Picking the Pivot

Although the algorithm as described works no matter which element is chosen as pivot, some choices are obviously better than others.

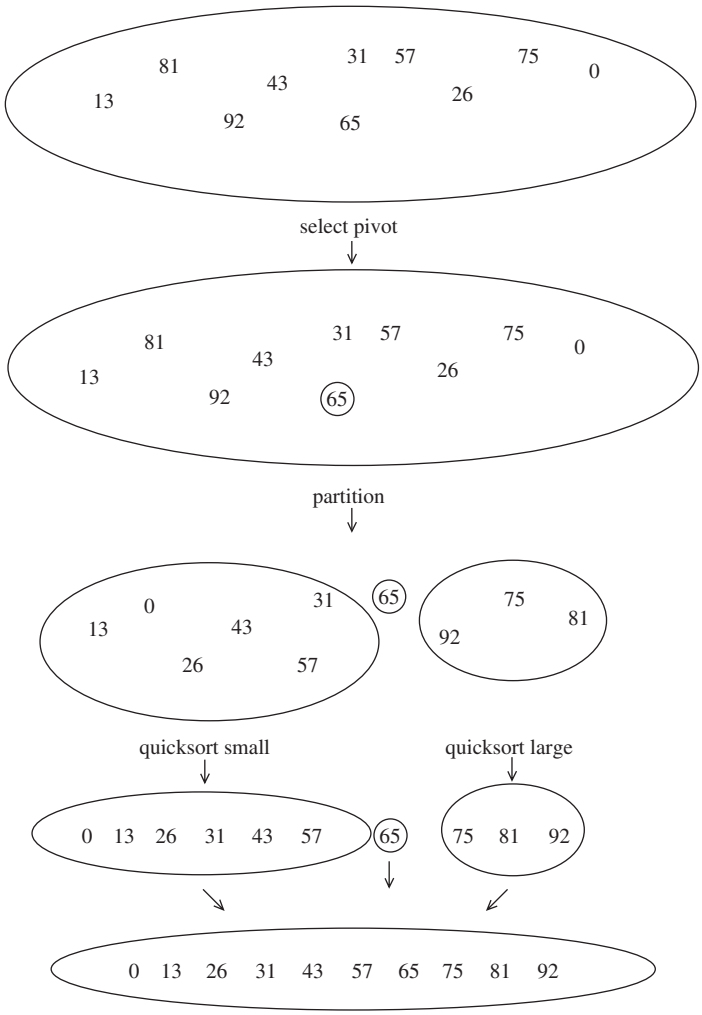

Figure 7.14 The steps of quicksort illustrated by example

A Wrong Way

The popular, uninformed choice is to use the first element as the pivot. This is acceptable if the input is random, but if the input is presorted or in reverse order, then the pivot provides a poor partition, because either all the elements go into S_1 or they go into S_2. Worse, this happens consistently throughout the recursive calls. The practical effect is that if the first element is used as the pivot and the input is presorted, then quicksort will take quadratic time to do essentially nothing at all, which is quite embarrassing. Moreover, presorted input (or input with a large presorted section) is quite frequent, so using the first element as pivot *is an absolutely horrible idea* and should be discarded immediately. An alternative is choosing the larger of the first two distinct elements as pivot, but this has

the same bad properties as merely choosing the first element. Do not use that pivoting strategy, either.

A Safe Maneuver

A safe course is merely to choose the pivot randomly. This strategy is generally perfectly safe, unless the random number generator has a flaw (which is not as uncommon as you might think), since it is very unlikely that a random pivot would consistently provide a poor partition. On the other hand, random number generation is generally an expensive commodity and does not reduce the average running time of the rest of the algorithm at all.

Median-of-Three Partitioning

The median of a group of N numbers is the $\lceil N/2 \rceil$th largest number. The best choice of pivot would be the median of the array. Unfortunately, this is hard to calculate and would slow down quicksort considerably. A good estimate can be obtained by picking three elements randomly and using the median of these three as pivot. The randomness turns out not to help much, so the common course is to use as pivot the median of the left, right, and center elements. For instance, with input $8, 1, 4, 9, 6, 3, 5, 2, 7, 0$ as before, the left element is 8, the right element is 0, and the center (in position $\lfloor (left + right)/2 \rfloor$) element is 6. Thus, the pivot would be $v = 6$. Using median-of-three partitioning clearly eliminates the bad case for sorted input (the partitions become equal in this case) and actually reduces the number of comparisons by 14%.

7.7.2 Partitioning Strategy

There are several partitioning strategies used in practice, but the one described here is known to give good results. It is very easy, as we shall see, to do this wrong or inefficiently, but it is safe to use a known method. The first step is to get the pivot element out of the way by swapping it with the last element. `i` starts at the first element and `j` starts at the next-to-last element. If the original input was the same as before, the following figure shows the current situation:

8	1	4	9	0	3	5	2	7	6
↑								↑	
i								j	

For now, we will assume that all the elements are distinct. Later on, we will worry about what to do in the presence of duplicates. As a limiting case, our algorithm must do the proper thing if *all* of the elements are identical. It is surprising how easy it is to do the *wrong* thing.

What our partitioning stage wants to do is to move all the small elements to the left part of the array and all the large elements to the right part. "Small" and "large" are, of course, relative to the pivot.

While `i` is to the left of `j`, we move `i` right, skipping over elements that are smaller than the pivot. We move `j` left, skipping over elements that are larger than the pivot. When `i` and `j` have stopped, `i` is pointing at a large element and `j` is pointing at a small element. If

`i` is to the left of `j`, those elements are swapped. The effect is to push a large element to the right and a small element to the left. In the example above, `i` would not move and `j` would slide over one place. The situation is as follows:

8	1	4	9	0	3	5	2	7	6
↑							↑		
i							j		

We then swap the elements pointed to by `i` and `j` and repeat the process until `i` and `j` cross:

After First Swap

2	1	4	9	0	3	5	8	7	6
↑							↑		
i							j		

Before Second Swap

2	1	4	9	0	3	5	8	7	6
			↑			↑			
			i			j			

After Second Swap

2	1	4	5	0	3	9	8	7	6
			↑			↑			
			i			j			

Before Third Swap

2	1	4	5	0	3	9	8	7	6
					↑	↑			
					j	i			

At this stage, `i` and `j` have crossed, so no swap is performed. The final part of the partitioning is to swap the pivot element with the element pointed to by `i`:

After Swap with Pivot

2	1	4	5	0	3	6	8	7	9
						↑			↑
						i			pivot

When the pivot is swapped with `i` in the last step, we know that every element in a position $p < i$ must be small. This is because either position p contained a small element

to start with, or the large element originally in position p was replaced during a swap. A similar argument shows that elements in positions $p > \texttt{i}$ must be large.

One important detail we must consider is how to handle elements that are equal to the pivot. The questions are whether or not `i` should stop when it sees an element equal to the pivot and whether or not `j` should stop when it sees an element equal to the pivot. Intuitively, `i` and `j` ought to do the same thing, since otherwise the partitioning step is biased. For instance, if `i` stops and `j` does not, then all elements that are equal to the pivot will wind up in S_2.

To get an idea of what might be good, we consider the case where all the elements in the array are identical. If both `i` and `j` stop, there will be many swaps between identical elements. Although this seems useless, the positive effect is that `i` and `j` will cross in the middle, so when the pivot is replaced, the partition creates two nearly equal subarrays. The mergesort analysis tells us that the total running time would then be $O(N \log N)$.

If neither `i` nor `j` stops, and code is present to prevent them from running off the end of the array, no swaps will be performed. Although this seems good, a correct implementation would then swap the pivot into the last spot that `i` touched, which would be the next-to-last position (or last, depending on the exact implementation). This would create very uneven subarrays. If all the elements are identical, the running time is $O(N^2)$. The effect is the same as using the first element as a pivot for presorted input. It takes quadratic time to do nothing!

Thus, we find that it is better to do the unnecessary swaps and create even subarrays than to risk wildly uneven subarrays. Therefore, we will have both `i` and `j` stop if they encounter an element equal to the pivot. This turns out to be the only one of the four possibilities that does not take quadratic time for this input.

At first glance it may seem that worrying about an array of identical elements is silly. After all, why would anyone want to sort 500,000 identical elements? However, recall that quicksort is recursive. Suppose there are 10,000,000 elements, of which 500,000 are identical (or, more likely, complex elements whose sort keys are identical). Eventually, quicksort will make the recursive call on only these 500,000 elements. Then it really will be important to make sure that 500,000 identical elements can be sorted efficiently.

7.7.3 Small Arrays

For very small arrays ($N \leq 20$), quicksort does not perform as well as insertion sort. Furthermore, because quicksort is recursive, these cases will occur frequently. A common solution is not to use quicksort recursively for small arrays, but instead use a sorting algorithm that is efficient for small arrays, such as insertion sort. Using this strategy can actually save about 15 percent in the running time (over doing no cutoff at all). A good cutoff range is $N = 10$, although any cutoff between 5 and 20 is likely to produce similar results. This also saves nasty degenerate cases, such as taking the median of three elements when there are only one or two.

7.7.4 Actual Quicksort Routines

The driver for quicksort is shown in Figure 7.15.

```
/**
 * Quicksort algorithm (driver).
 */
template <typename Comparable>
void quicksort( vector<Comparable> & a )
{
    quicksort( a, 0, a.size( ) - 1 );
}
```

Figure 7.15 Driver for quicksort

The general form of the routines will be to pass the array and the range of the array (`left` and `right`) to be sorted. The first routine to deal with is pivot selection. The easiest way to do this is to sort `a[left]`, `a[right]`, and `a[center]` in place. This has the extra advantage that the smallest of the three winds up in `a[left]`, which is where the partitioning step would put it anyway. The largest winds up in `a[right]`, which is also the correct place, since it is larger than the pivot. Therefore, we can place the pivot in `a[right - 1]` and initialize `i` and `j` to `left + 1` and `right - 2` in the partition phase. Yet another benefit is that because `a[left]` is smaller than the pivot, it will act as a sentinel for `j`. Thus, we do not need to worry about `j` running past the end. Since `i` will stop on elements equal to the pivot, storing the pivot in `a[right-1]` provides a sentinel for `i`. The code in

```
/**
 * Return median of left, center, and right.
 * Order these and hide the pivot.
 */
template <typename Comparable>
const Comparable & median3( vector<Comparable> & a, int left, int right )
{
    int center = ( left + right ) / 2;

    if( a[ center ] < a[ left ] )
        std::swap( a[ left ], a[ center ] );
    if( a[ right ] < a[ left ] )
        std::swap( a[ left ], a[ right ] );
    if( a[ right ] < a[ center ] )
        std::swap( a[ center ], a[ right ] );

        // Place pivot at position right - 1
    std::swap( a[ center ], a[ right - 1 ] );
    return a[ right - 1 ];
}
```

Figure 7.16 Code to perform median-of-three partitioning